4 Markdown Files Every Vibe Coder Should Know

A practical starter guide for giving AI coding agents better context, cleaner structure, and fewer reasons to guess.

The idea: Instead of putting every instruction into one giant prompt, keep important project context in dedicated Markdown files. Your coding agent can reference them throughout the build.

1. PRD.md

Product Requirements Document

Defines what you're building, who it's for, why it matters, and what the product needs to do.

Recommended structure

- **Product overview —** Name, one-line description, product vision.
- **Problem —** The user problem the product is solving.
- **Goal —** The outcome the product should create.
- **Target users —** Who the product is designed for.
- **Core features —** The essential functionality for the first version.
- **User flows —** The key journeys users need to complete.
- **Requirements —** Functional, UX, performance, and platform requirements.
- **Success metrics —** How you'll know the product is working.
- **Out of scope —** What should not be built yet.

Starter template

```
# Product Requirements Document

## Product Overview
Product: AI Portfolio Builder
Goal: Help creators launch a portfolio in under 10 minutes.

## Problem
Creators need a faster way to build professional portfolios
without spending hours designing or coding.

## Target Users
- Creators
- Freelancers
- Students
- Job seekers

## Core Features
- AI website generation
- Customizable templates
- Project showcase
- One-click publishing
- Analytics
```

```
## Success Metrics
- Portfolio completion rate
- Publish rate
- User satisfaction
```

2. AGENTS.md

AI Agent Instructions

Tells your AI coding agent how it should work inside the project, including rules, conventions, commands, and boundaries.

Recommended structure

- **Project context —** A short explanation of the product and stack.
- **Before you start —** Files the agent must read before changing code.
- **General rules —** Project-wide development rules.
- **Code guidelines —** Component, naming, TypeScript, and reuse expectations.
- **Design rules —** How the agent should follow the design system.
- **Security rules —** What the agent must never expose or bypass.
- **Commands —** Install, development, build, lint, and test commands.
- **Boundaries —** Things the agent should not change without approval.

Starter template

```
# Agent Instructions

## Before You Start
- Read PRD.md
- Read DESIGN_SYSTEM.md
- Read ARCHITECTURE.md
- Inspect existing components before creating new ones

## General Rules
- Use TypeScript
- Reuse existing components
- Keep components modular
- Follow the existing folder structure
- Ask before making major architectural changes

## Security
- Never expose API keys
- Keep secrets in environment variables
- Validate user input
- Verify authorization server-side

## Commands
npm install
npm run dev
npm run build
npm run lint
npm run test
```

3. DESIGN_SYSTEM.md

Design System

Defines the visual rules your coding agent should follow so generated screens feel like one product instead of unrelated pages.

Recommended structure

- **Brand direction —** The visual personality and overall feel.
- **Color tokens —** Primary, secondary, background, surface, text, and states.
- **Typography —** Fonts, sizes, weights, and line heights.
- **Spacing —** A consistent spacing scale.
- **Radius & shadows —** Rules for corners, borders, and elevation.
- **Components —** Buttons, inputs, cards, navigation, modals, etc.
- **States —** Hover, focus, active, disabled, loading, empty, and error states.
- **Responsive rules —** Breakpoints and mobile/tablet/desktop behavior.
- **Accessibility —** Contrast, focus states, semantic elements, and keyboard behavior.

Starter template

```
# Design System

## Direction
Clean, modern, minimal, and creator-focused.

## Colors
--background: #FAFAFA
--text: #0A0A0A
--primary: #FF3EBF
--secondary: #6366F1
--success: #22C55E
--error: #EF4444

## Typography
Font: Inter
H1: 64px / 72px / Bold
H2: 48px / 56px / Semibold
Body: 16px / 24px / Regular

## Spacing
Use an 8px spacing system:
8, 16, 24, 32, 48, 64

## Components
- Primary / Secondary / Ghost buttons
- Inputs include label, error, focus and disabled states
- Reuse existing components before creating new ones

## Responsive
Mobile: < 640px
Tablet: 640–1024px
Desktop: > 1024px
```

4. ARCHITECTURE.md

System Architecture

Explains how the app is structured, which technologies it uses, and how the frontend, backend, database, and external services connect.

Recommended structure

- **System overview —** A simple map of the major parts of the product.
- **Tech stack —** Frameworks, database, auth, payments, email, analytics, etc.
- **Project structure —** What each important folder is responsible for.
- **Data flow —** How requests move through the system.
- **Database & storage —** Where product data and files live.
- **External services —** Third-party APIs and integrations.
- **Deployment —** Where and how the application is deployed.
- **Scalability notes —** Caching, queues, monitoring, background jobs, and future considerations.

Starter template

```
# Architecture

## System Overview
User
  ↓
Next.js Frontend
  ↓
Server / API Layer
  ↓
Supabase Database

External Services:
- Clerk → Authentication
- Stripe → Payments
- Resend → Email
- PostHog → Analytics
- Sentry → Error monitoring
- Vercel → Deployment

## Project Structure
/app          # Routes and pages
/components   # Reusable UI
/lib          # Utilities and configuration
/services     # API and external services
/types        # TypeScript types
/public       # Static assets
/docs         # Project documentation

## Data Flow
1. User interacts with the UI
2. Frontend sends validated request
3. Server verifies authentication/authorization
4. Database or external service processes request
5. Response returns to the UI
```

```
6. UI handles success, loading and error states
```

How to use these files with an AI coding agent

1. Create a /docs folder in your project (or keep AGENTS.md at the project root if your workflow expects it there).
2. Fill the files with decisions that are specific to your project. Avoid vague instructions like “make it modern.”
3. At the beginning of a build, tell the agent which files to read before writing code.
4. When a product decision changes, update the relevant Markdown file so the project context stays current.
5. Keep the files focused. PRD.md owns product requirements; DESIGN_SYSTEM.md owns UI rules; ARCHITECTURE.md owns technical structure.
6. Treat the documents as living context, not one-time setup files.

Copy/paste kickoff prompt

```
Before writing code, read:
- PRD.md
- AGENTS.md
- DESIGN_SYSTEM.md
- ARCHITECTURE.md

Use these files as the source of truth for product scope,
coding rules, UI decisions, and system architecture.

Before creating a new component or pattern, check whether
one already exists. Do not change the architecture or add
new dependencies unless they are necessary.

If the files conflict or an important decision is missing,
ask for clarification before making a major assumption.
```

Quick rule of thumb

PRD.md = What are we building? **AGENTS.md** = How should the AI work? **DESIGN_SYSTEM.md** = How should it look? **ARCHITECTURE.md** = How should it connect?